Faculty of Engineering and Architectural Science

Department of Computer and Electrical Engineering

Course Number	**COE718**
Course Title	**Embedded Systems Design**
Semester/Year	**Fall 2025**
Instructor	**Gul N. khan**
Teaching Assistant	

Lab Report No.	3a

Report Title	**RTX based Multitasking with Round-Robin Scheduli**

Section No.	05
Due Date of Lab Performance	Oct 8, 6 PM
Report Submission Date	

Name	Student ID	Signature*
Hamza Malik	501112545	H.M

**By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: http://www.ryerson.ca/senate/policies/pol60.pdf. .*

Abstract

This lab explored the concept of multitasking in embedded systems using the Keil RTX Real-Time Operating System (RTOS). The objective was to understand thread scheduling through round-robin execution, configure the RTX environment in uVision, and analyze the performance of multiple concurrent threads. Using the LPC1768 microcontroller and Keil simulator tools such as the Watch Window, Performance Analyzer, and Event Viewer, two threads were created and executed in alternating time slices of 10 ms. The behavior of the system was analyzed through task switching patterns, CPU utilization, and timing visualization. The experiment was extended by modifying scheduler parameters and observing the impact on system timing and responsiveness. The lab demonstrated the efficiency and predictability of RTOS-based scheduling and its application in multitasking embedded systems.

Introduction

In embedded systems, multitasking is essential for running several processes concurrently without sacrificing timing precision or system stability. Real-Time Operating Systems (RTOS) like **Keil RTX** provide deterministic scheduling and task management capabilities that make this possible. The aim of this lab was to introduce the fundamentals of real-time multitasking using RTX by developing and analyzing simple threaded applications. The **round-robin scheduling** technique was implemented, where each task receives an equal time slice to execute before control is passed to the next task. This ensures fair CPU utilization across all tasks. The lab involved creating a project in Keil uVision using the **LPC1768 microcontroller**, configuring the RTX kernel timer and tick intervals, and observing the execution of two threads. Tools like the **Watch Window**, **Performance Analyzer**, and **Event Viewer** were used to visualize variable changes, thread execution patterns, and overall CPU activity. The experiment provided hands-on understanding of thread management, kernel initialization, and the role of the idle thread in measuring CPU utilization.

Experimental Results

After configuring the RTX environment in **uVision**, the lab project was compiled and executed using the simulator. The main program created two threads (Thread1 and Thread2) using the osThreadDef() and osThreadCreate() functions. Each thread incremented a separate variable (counta and countb) continuously in a loop.

1. Watch Window Results

In the Watch Window, both counta and countb were observed to increment alternately, confirming that the round-robin scheduler was switching between the two threads every 10 milliseconds as configured in RTX_Conf_CM.c. When one variable increased, the other paused, showing proper context switching.

2. Performance Analyzer

The Performance Analyzer showed both threads executing with nearly equal CPU utilization. The timeline confirmed a consistent alternation between Thread1 and Thread2, validating the fairness of round-robin scheduling. Minor timing variations appeared when system interrupts occurred, but the overall cycle remained stable.

3. Event Viewer

The Event Viewer provided a graphical representation of thread activity. Blue blocks represented each thread's execution period, while gaps indicated context switches. When hovering over the thread slices, the measured execution time matched the 10 ms round-robin time slice. When the time slice was changed to 15 ms in RTX_Conf_CM.c, the Event Viewer accurately reflected the new scheduling intervals.

4. System and Thread Viewer

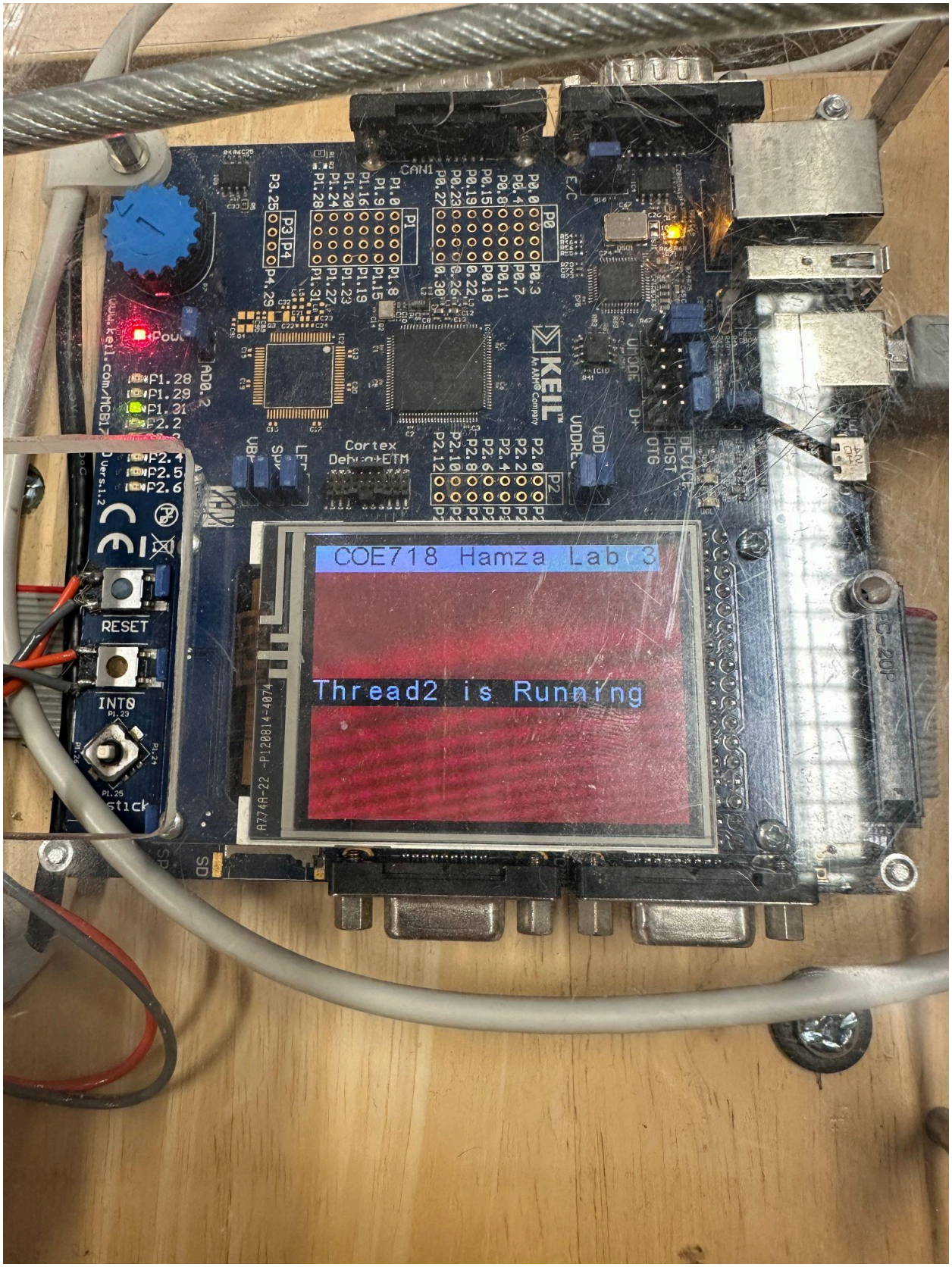
The System and Thread Viewer displayed the list of active threads, their priorities, and states. The osTimerThread() executed first to initialize timing functions, followed by Thread1 and Thread2. When an idle task was introduced (countIDLE variable), it showed that under full thread activity, CPU utilization reached 100%, meaning no idle time was available during thread execution.

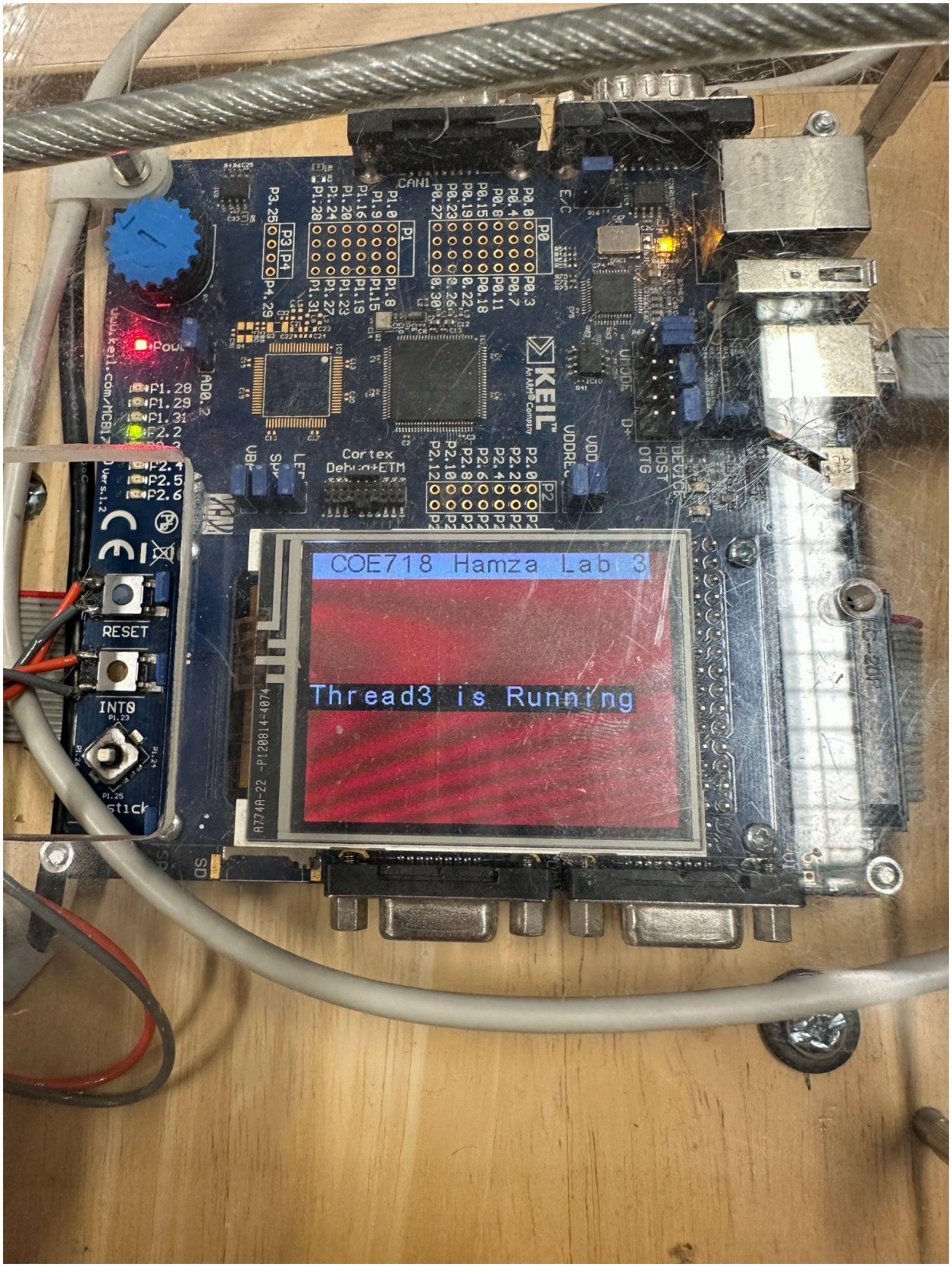
Discussion

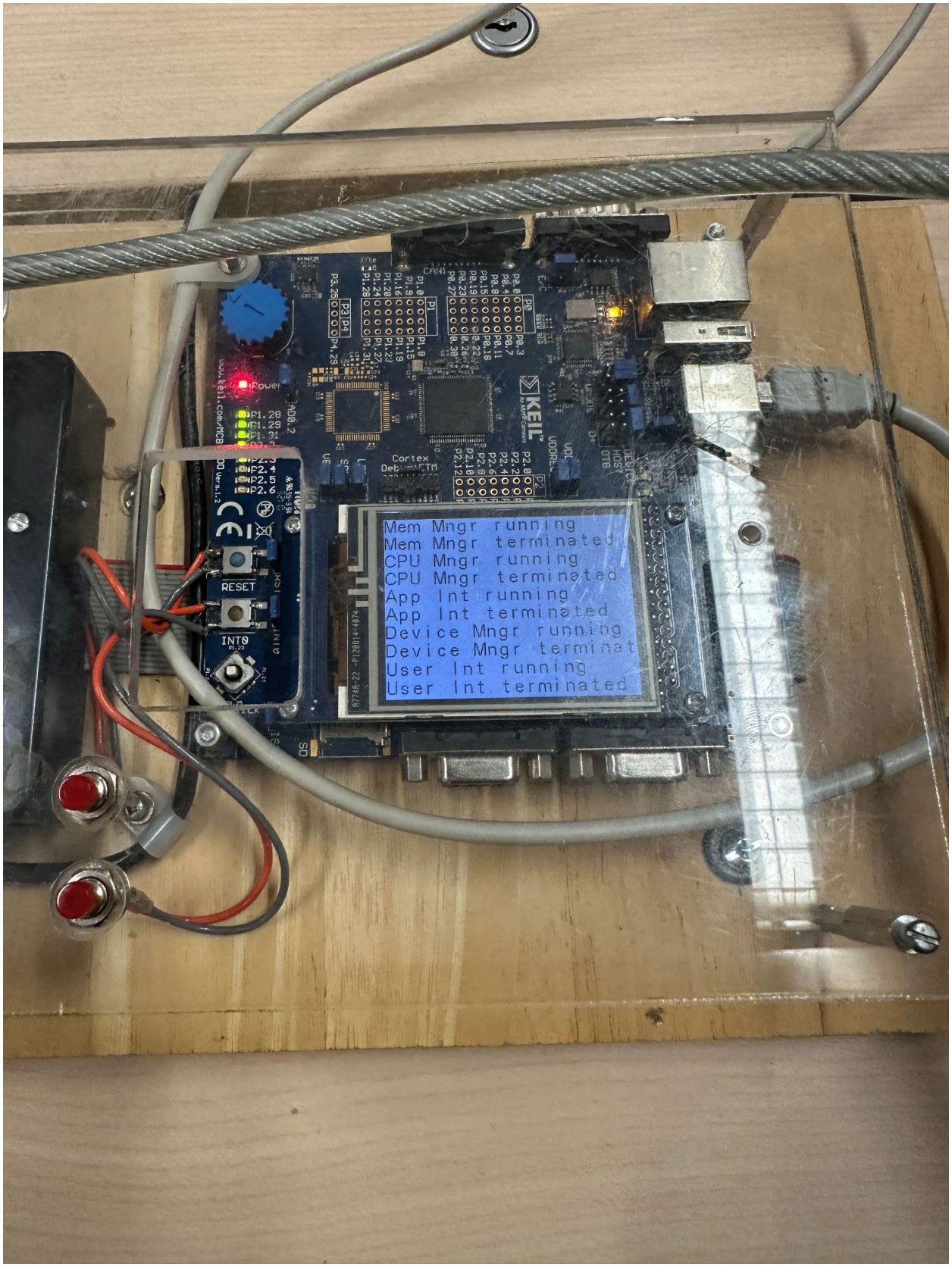
This lab effectively demonstrated how **Keil RTX** handles multitasking using **preemptive round-robin scheduling**. Each thread was given an equal and fixed time slice, ensuring fair access to the processor. Through tools such as the Watch Window and Event Viewer, students could visualize the internal operation of the RTOS scheduler. The experiment showed that by adjusting the time-slice value, the responsiveness of the system changes. A smaller time slice leads to more frequent context switching, improving responsiveness but increasing overhead. Conversely, a larger time slice reduces switching overhead but increases the waiting time for lower-priority tasks. The experiment also illustrated how the idle thread indicates processor efficiency; when active threads occupy the full CPU, the idle counter remains constant at zero, confirming total CPU utilization. One of the key takeaways was understanding that although round-robin scheduling ensures equality among tasks, it is not ideal for real-time applications where certain threads have strict timing deadlines. In such cases, priority-based scheduling would be more suitable. However, for tasks of equal importance—like monitoring joystick inputs, reading sensors, or updating graphs—round-robin scheduling provides a balanced and predictable behavior.

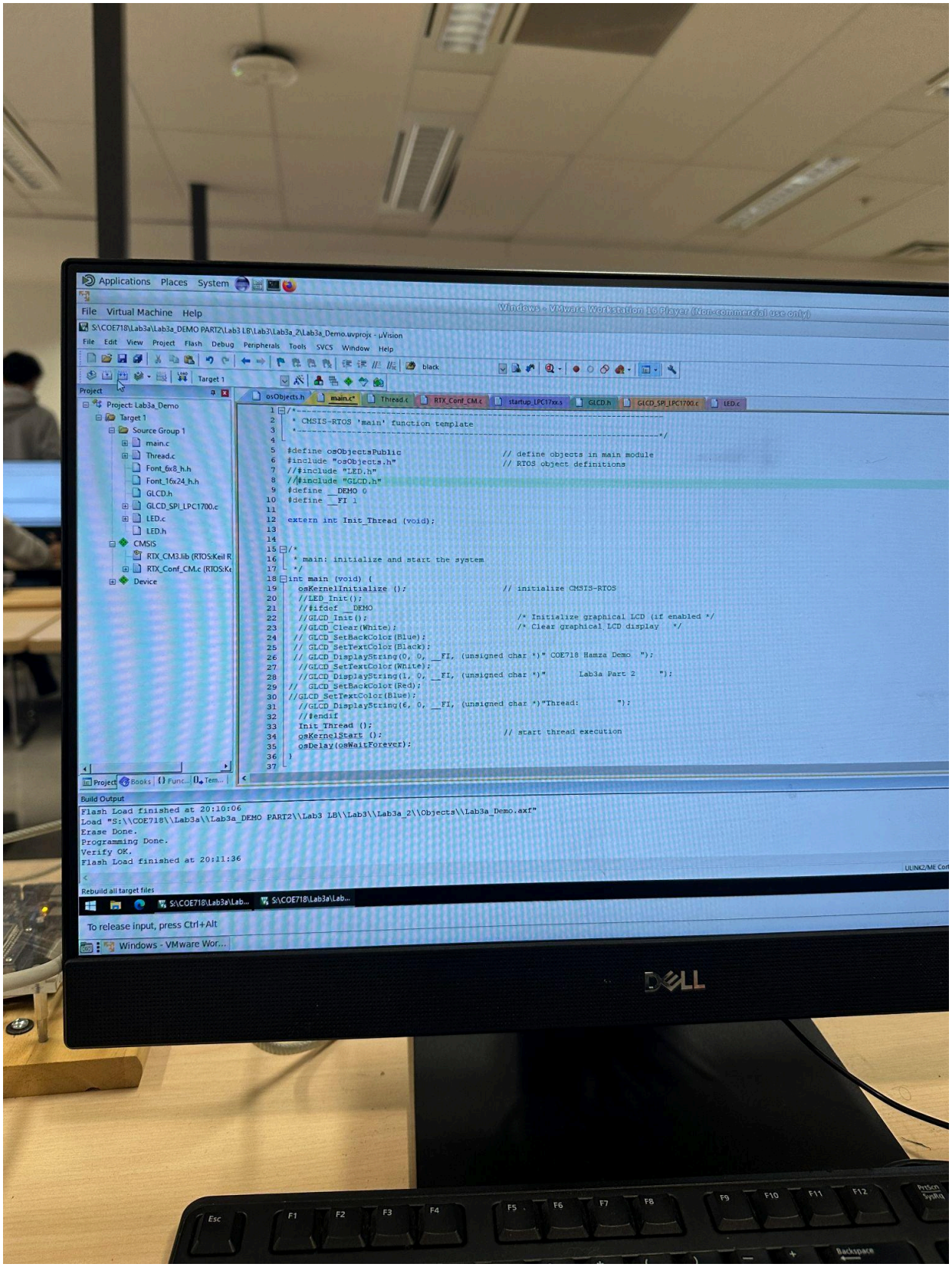
Conclusion

Through this lab, the fundamentals of multitasking using the **Keil RTX RTOS** were successfully explored. The creation, configuration, and execution of multiple threads provided practical insight into real-time scheduling. Tools such as the Watch Window, Performance Analyzer, and Event Viewer made it possible to visualize how tasks are executed and switched in real-time. The experiment confirmed that round-robin scheduling allows fair task execution, though it may not always optimize performance for tasks with differing priorities. The idle thread analysis highlighted that with continuous thread execution, CPU utilization can reach full capacity. Overall, this lab enhanced understanding of **thread synchronization, timing control, and RTOS kernel behavior**, forming a strong foundation for more advanced embedded system design involving interrupts, mutexes, and event-driven task management.


COE718 Hamza Lab 3
Thread2 is Running
RESET
INT0
CAN1
KEIL
Cortex
HOST
DEVICE
OTG


COE718 Hamza Lab 3
Thread3 is Running
RESET
INT0
KEIL
An ARM® Company
Cortex
Debug+ETM
CAN1
USB
HOST
OTG
DEVICE


Mem Mngr running
Mem Mngr terminated
CPU Mngr running
CPU Mngr terminated
App Int running
App Int terminated
Device Mngr running
Device Mngr terminat
User Int running
User Int terminated
RESET
INT0
KEIL


Applications Places System
File Virtual Machine Help
Windows - VMware Workstation 16 Player (Non-commercial use only)
S:\COE718\Lab3a\Lab3a_DEMO PART2\Lab3 LB\Lab3\Lab3a_2\Lab3a_Demo.uvprojx - µVision
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Project: Lab3a_Demo
Target 1
Source Group 1
main.c
Thread.c
Font_6x8_h.h
Font_16x24_h.h
GLCD.h
GLCD_SPI_LPC1700.c
LED.c
LED.h
CMSIS
RTX_CM3.lib (RTOS:Keil R
RTX_Conf_CM.c (RTOS:Ke
Device
osObjects.h
main.c*
Thread.c
RTX_Conf_CM.c
startup_LPC17xx.s
GLCD.h
GLCD_SPI_LPC1700.c
LED.c
/*--
* CMSIS-RTOS 'main' function template
--/
#define osObjectsPublic // define objects in main module
#include "osObjects.h" // RTOS object definitions
//#include "LED.h"
//#include "GLCD.h"
#define __DEMO 0
#define __FI 1
extern int Init_Thread (void);
/*
* main: initialize and start the system
*/
int main (void) {
osKernelInitialize (); // initialize CMSIS-RTOS
//LED_Init();
//#ifdef __DEMO
//GLCD_Init(); /* Initialize graphical LCD (if enabled */
//GLCD_Clear(White); /* Clear graphical LCD display */
// GLCD_SetBackColor(Blue);
// GLCD_SetTextColor(Black);
// GLCD_DisplayString(0, 0, __FI, (unsigned char *)" COE718 Hamza Demo ");
//GLCD_SetTextColor(White);
//GLCD_DisplayString(1, 0, __FI, (unsigned char *)" Lab3a Part 2 ");
// GLCD_SetBackColor(Red);
//GLCD_SetTextColor(Blue);
//GLCD_DisplayString(6, 0, __FI, (unsigned char *)"Thread: ");
//#endif
Init_Thread ();
osKernelStart (); // start thread execution
osDelay(osWaitForever);
}
Build Output
Flash Load finished at 20:10:06
Load "S:\\COE718\\Lab3a\\Lab3a_DEMO PART2\\Lab3 LB\\Lab3\\Lab3a_2\\Objects\\Lab3a_Demo.axf"
Erase Done.
Programming Done.
Verify OK.
Flash Load finished at 20:11:36
Rebuild all target files
To release input, press Ctrl+Alt
Windows - VMware Wor...
DELL